

Computer Graphics – October 14th, 2021

Name: _____

Seat Number: _____

Matriculation Number: _____

Please read the following instructions carefully:

- You have sixty (60) minutes to complete the exam.
- You can answer in German or in English.
- Check that the exam is complete (22 pages on 11 sheets) and readable!
- Please use a blue or black ballpen, not a red or green one. Do not use a pencil!
- Only text on the stapled sheets will be evaluated.
- If needed, we staple additional blank sheets.
- Only writing material is allowed!

I have read and understood the instructions above.

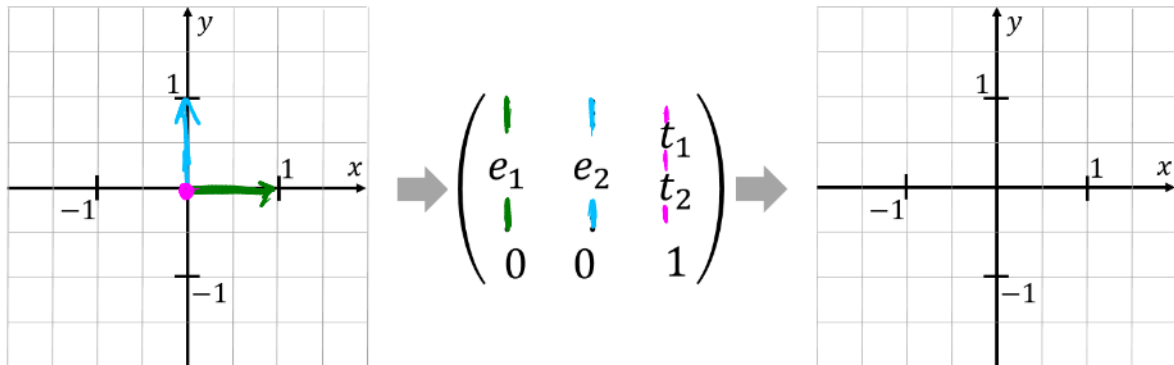
Erlangen, October 14th, 2021 _____
Signature

1	2	3	4	5	6	ALL
12	14	10	10	9	5	60

Grade:

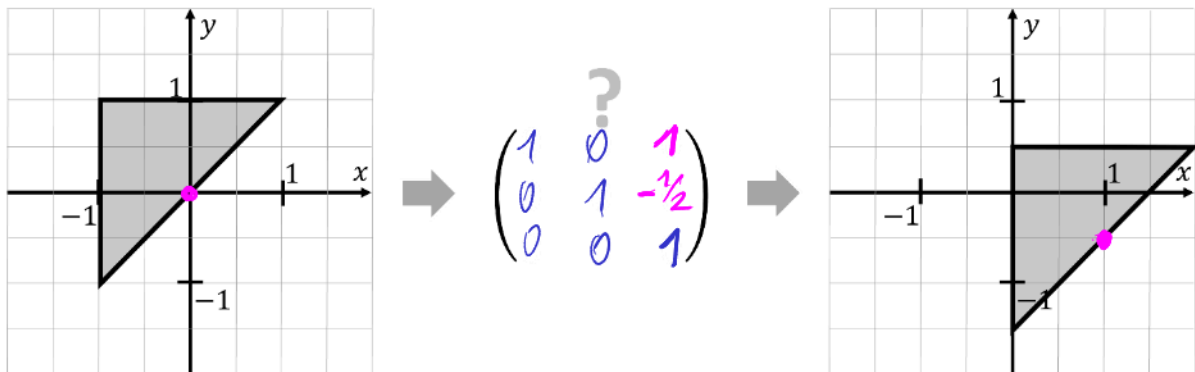
Assignment 1: Affine Mappings in 2D [12 points]

Affine mappings in **2D** can be expressed using **homogeneous coordinates** and 3×3 matrices with the following structure, where $(\vec{e}_1, \vec{e}_2)$ is the linear part (image of base vectors) and (t_x, t_y) is the translation:

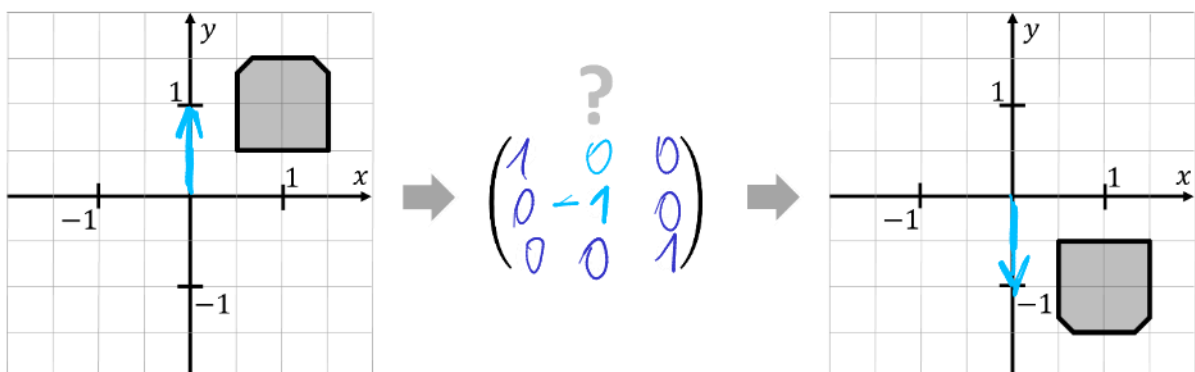


Each of the sub-assignments a) to e) deals with one mapping (in the coordinate system depicted above) and consists of two images (before and after mapping) and one matrix that has the desired effect. Complete each of the sub-assignments by **filling in the missing part** marked with a question mark.

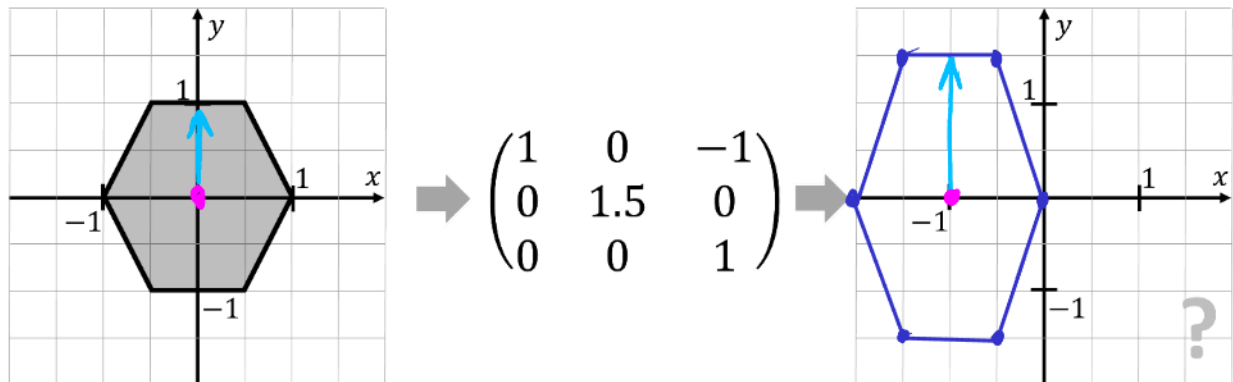
a) [1 P]



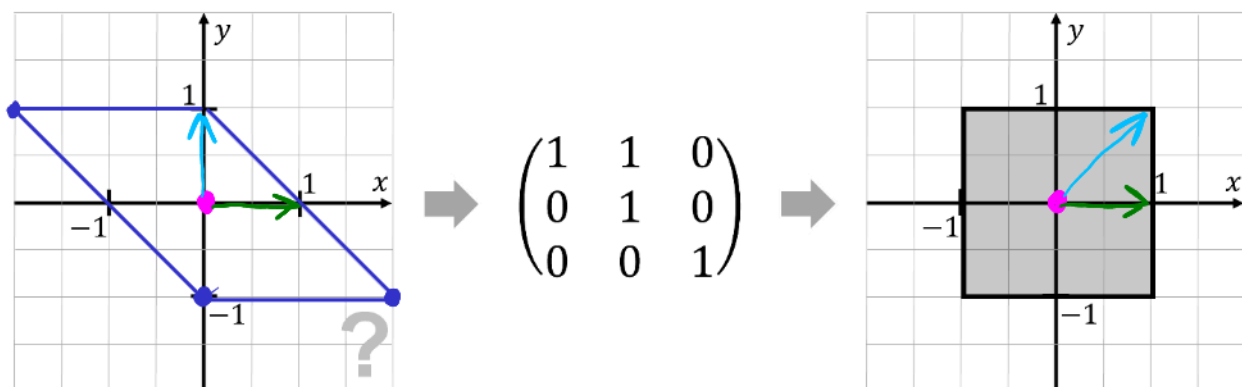
b) [1 P]



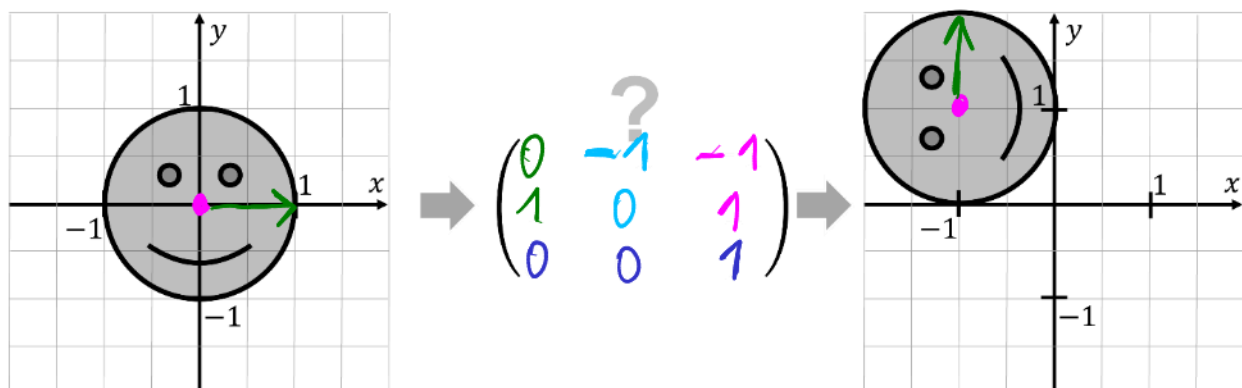
c) [2 P]



d) [2 P]



e) [2 P]



f) [4 P] Create valid 2D transformation matrices from the given templates on the right, while adhering to the transformation class constraints on the left. If such a matrix is impossible to create from the template, check the box in the middle instead.

1) Linear, but not rigid

☐ impossible

$$\begin{pmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

2) Rigid, but not linear

☐ impossible

$$\begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix}$$

3) Rigid and linear

☐ impossible

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

4) Projective, but not affine

☒ impossible

$$\begin{pmatrix} & 0 & \\ 0 & & \\ 0 & 0 & 1 \end{pmatrix}$$

Assignment 2: Perspective Projection & Texture Mapping [14 points]

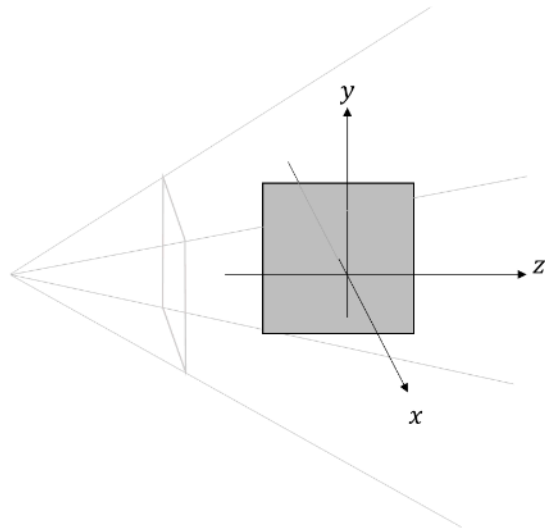
Given is the **square** $ABCD$ with the four vertices

$$A = (-1, -1, -1), B = (-1, 1, -1),$$

$$C = (-1, 1, 1), D = (-1, -1, 1).$$

We project the square **perspectively** using the following homogeneous matrix:

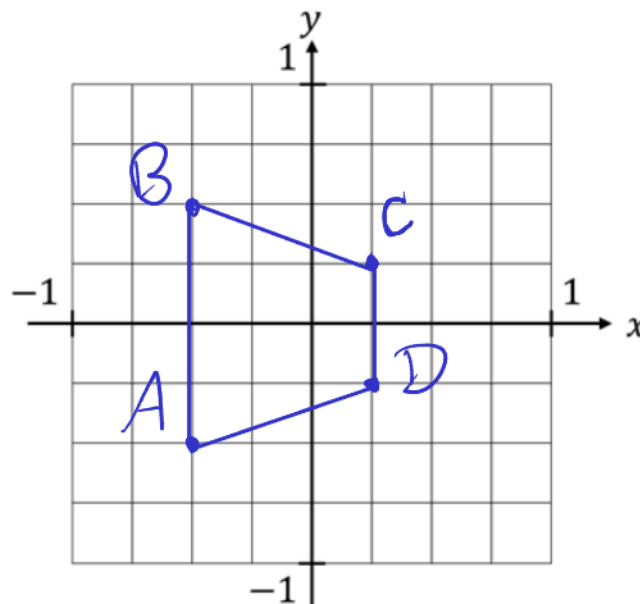
$$M = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 3 \end{pmatrix}$$



- a) **[4 P]** Map the vertices to image space using matrix M and fill in the table with the mapping results. For all vertices, give both their two-dimensional image coordinates and their depth value, both **after dehomogenization**!

Vertex	Image coordinates $\in [-1, 1]^2$	Depth $\in [-1, 1]$
A	$-\frac{1}{2}, -\frac{1}{2}$	$-\frac{1}{2}$
B	$-\frac{1}{2}, \frac{1}{2}$	$-\frac{1}{2}$
C	$\frac{1}{4}, \frac{1}{4}$	$-\frac{1}{4}$
D	$\frac{1}{4}, -\frac{1}{4}$	$-\frac{1}{4}$

- b) **[1 P]** Sketch the resulting shape in image space into the following 8x8 pixel grid:



② a) $\tilde{A} = \begin{pmatrix} A_1 \\ A_2 \\ A_3 \\ 1 \end{pmatrix}$ "homogenized"

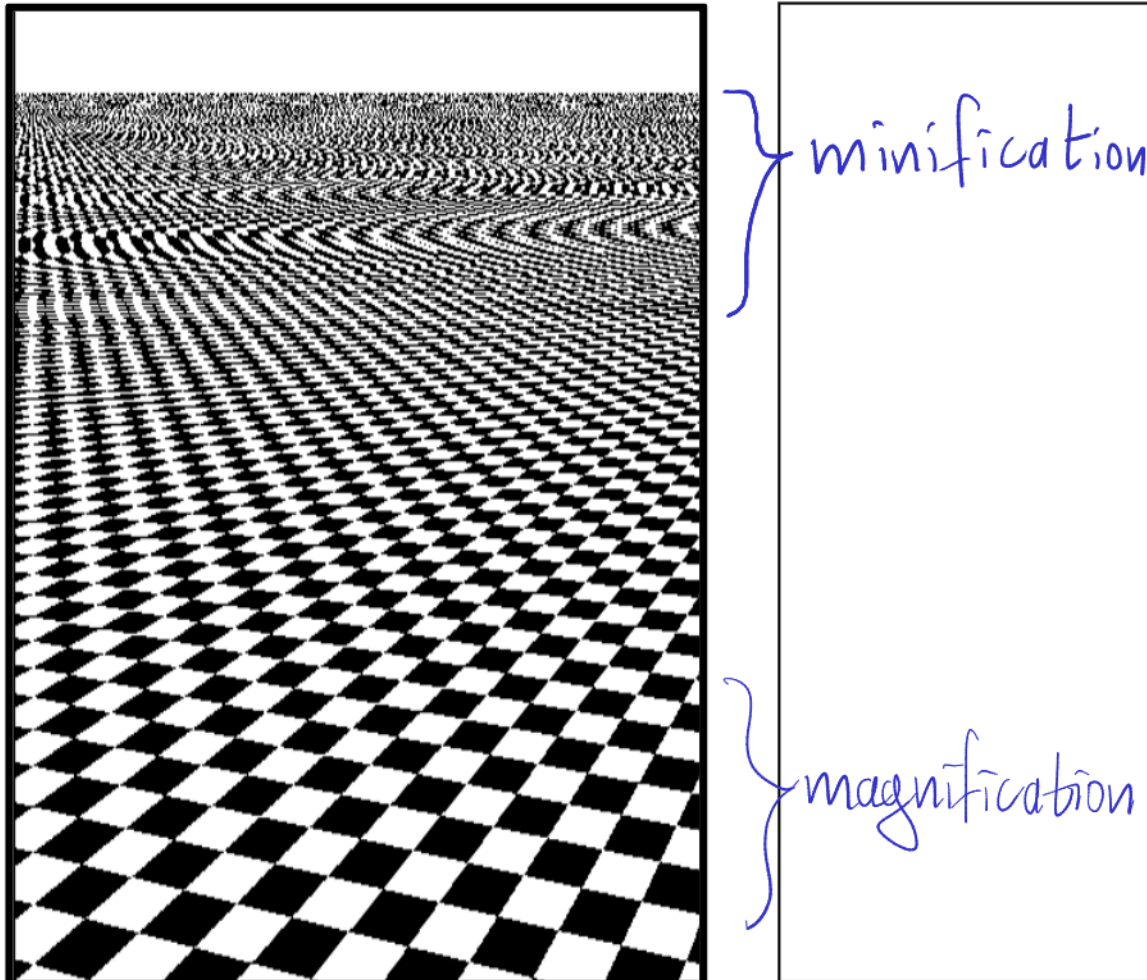
$$M\tilde{A} = \begin{pmatrix} -1 \\ -1 \\ -1 \\ 2 \end{pmatrix} \xrightarrow{\text{perspective divide}} \begin{pmatrix} -1/w \\ -1/w \\ -1/w \end{pmatrix} = \begin{pmatrix} -1/2 \\ -1/2 \\ -1/2 \end{pmatrix} = \begin{pmatrix} x \\ y \\ \text{depth} \end{pmatrix}$$

$$M\tilde{B} = \begin{pmatrix} -1 \\ 1 \\ -1 \\ 2 \end{pmatrix} \xrightarrow{w=2} \begin{pmatrix} -1/2 \\ 1/2 \\ -1/2 \end{pmatrix}$$

$$M\tilde{C} = \begin{pmatrix} 1 \\ 1 \\ -1 \\ 4 \end{pmatrix} \rightarrow \begin{pmatrix} 1/4 \\ 1/4 \\ -1/4 \end{pmatrix}$$

$$M\tilde{D} = \begin{pmatrix} 1 \\ -1 \\ -1 \\ 4 \end{pmatrix} \rightarrow \begin{pmatrix} 1/4 \\ -1/4 \\ -1/4 \end{pmatrix}$$

- c) [2 P] In the image below, you can see a plane, textured with a checkerboard texture and looked at under an angle. In the box on the right, roughly state/show where texture **minification** or texture **magnification** appears for this plane.

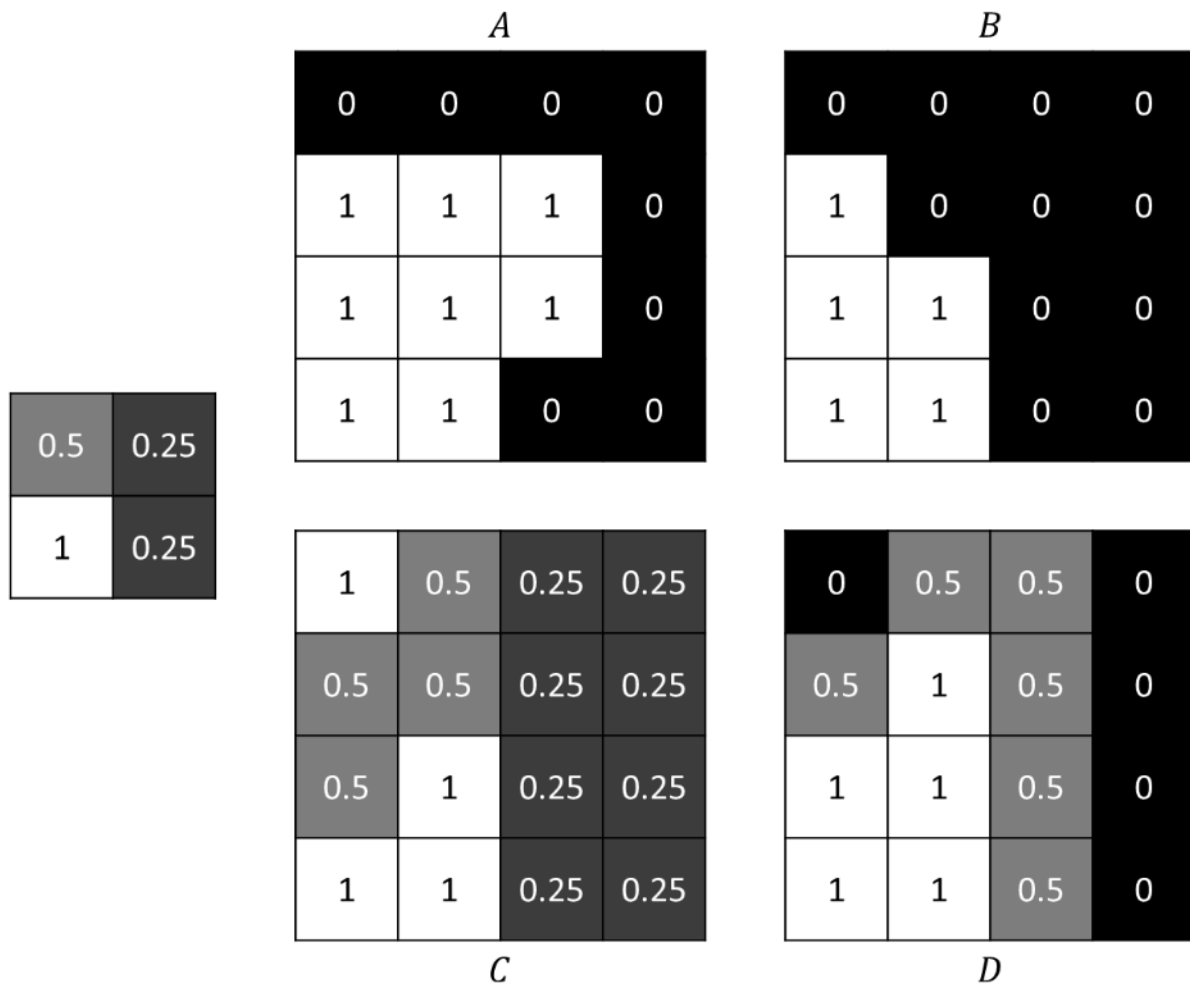


- d) [2 P] For both texture minification and magnification, shortly state **which visual artifacts** they can result in and what can be done **to reduce or avoid** them.

minification: - new patterns appear
- if pixel footprint > texel size then average over texture
→ use a MIP-map

magnification: - reveals individual texels
- use linear interpolation

- e) [2 P] In the left side of the image below, you can see the 2x2 level of a **MIP map pyramid** belonging a 4x4 texture. For each of the four texture candidates *A*, *B*, *C* and *D*, state **whether** it could be the 4x4 texture the 2x2 level on the left belongs to!



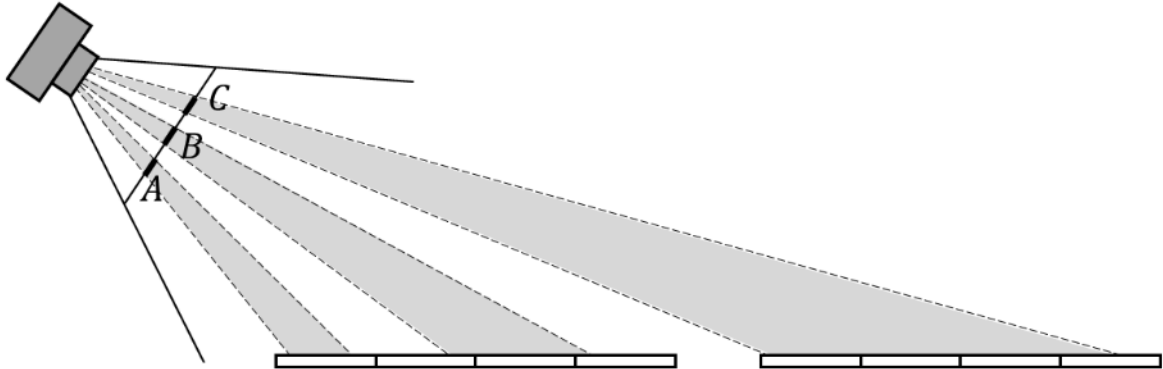
A: possible

B: not possible

C: not possible

D: possible

- f) **[3 P]** In the 2D example below, a camera captures two 1D lines (at the bottom). The two lines should both be textured with a **1D texture of 4 pixels**, including a **MIP map pyramid** with two additional levels of 2 and 1 texel(s), respectively. For each of the three pixels *A*, *B* and *C*, state which levels of the MIP map pyramid (4 pixels, 2 pixels, 1 pixel) are used in the case of trilinear interpolation!



A: 4 pixels

B: interpolation of "4 pixels" and "2 pixels"

C: interpolation of "2 pixels" and "1 pixel"

Assignment 3: Line Rasterization [10 points]

In the lecture, you learned about the Bresenham algorithm for line rasterization:

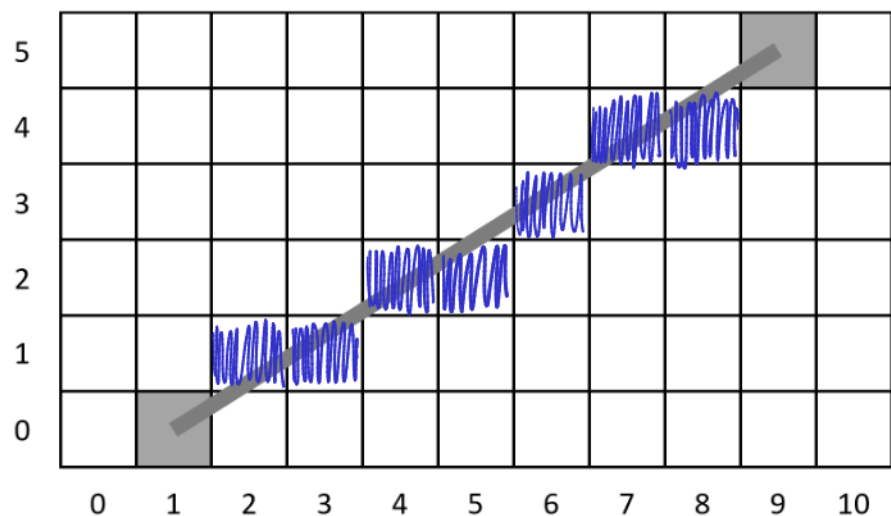
```
function line(int x0, int y0, int x1, int y1)
{
    int x = x0, y = y0
    int Δx = x1 - x0, Δy = y1 - y0
    int D = Δx - 2Δy
    int ΔDE = -2Δy, ΔDNE = 2(Δx - Δy)
    while (x <= x1) {
        draw_pixel(x,y)
        x = x + 1
        if (D < 0) {
            y = y + 1
            D = D + ΔDNE
        }
        else
            D = D + ΔDE
    }
}
```

- a) [6 P] Rasterize the line from (1, 0) to (9, 5) using this algorithm. Use the table below and fill in the variable values *at the beginning* of each loop iteration. Mark the set pixels in the grid below.

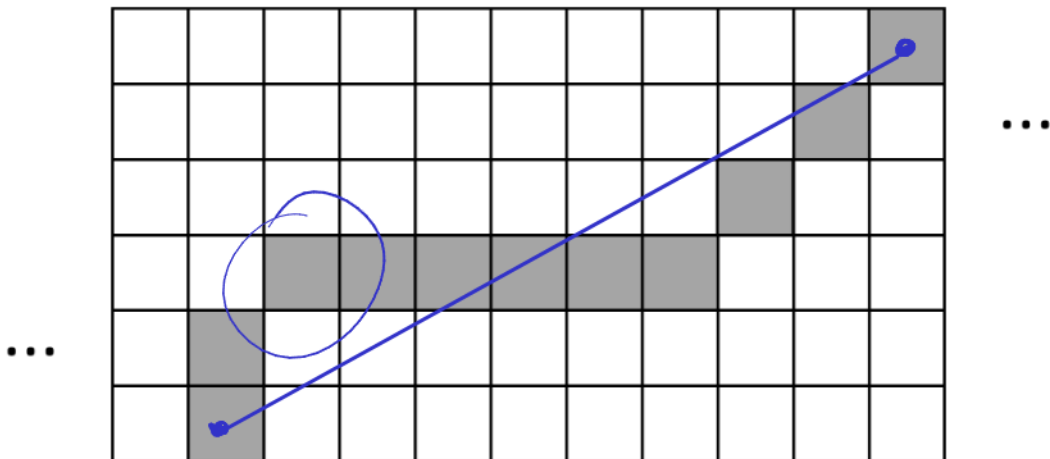
$$\Delta x = 8, \Delta y = 5$$

$$\Delta DE = -10 \quad \Delta DNE = 6$$

x	y	D
1	0	-2
2	1	4
3	1	-6
4	2	0
5	2	-10
6	3	-4
7	4	2
8	4	-8
9	5	-2



- b) [2 P] Briefly state **two** reasons why this line **cannot** be a result of the Bresenham algorithm.



Some filled pixel don't touch the line through the first and last visible pixel

- c) [2 P] What kind of lines does the unmodified Bresenham algorithm above produce for cases where $\Delta y > \Delta x$? Hint: Have a look at the resulting values for the ΔDE and ΔDNE variables.

$$\Delta DE = -2\Delta y$$

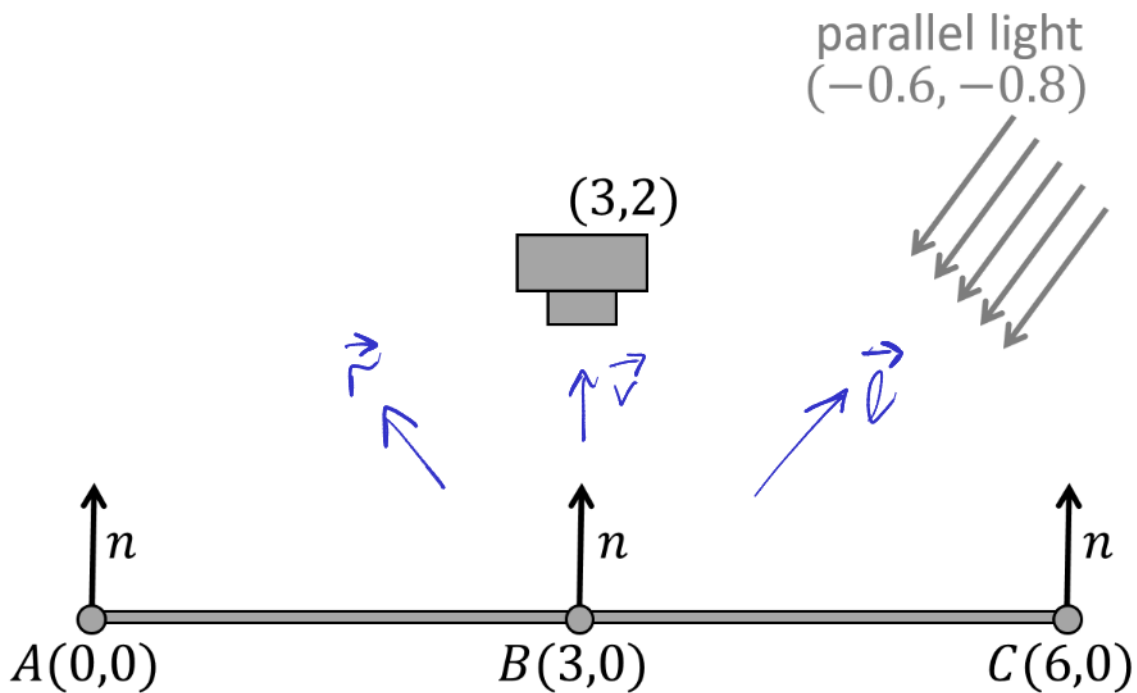
$$\Delta DNE = 2(\Delta x - \Delta y) = 2\Delta x - 2\Delta y$$

if $\Delta x > 0$ and $\Delta y > 0$ then both, ΔDE and ΔDNE are negative

$\Rightarrow D$ will stay negative $\Rightarrow$ a line with inclination $m=1$ will be drawn

Assignment 4: Phong Lighting & Shading [10 points]

The image below shows three points A , B and C belonging to a surface with the common normal vector $\mathbf{n} = (0, 1)$. The surface is lit by a **parallel light** with rays in direction $(-0.6, -0.8)$ and captured by a camera located at $(3, 2)$.



- a) [3 P] For the point B , write down the respective light vector l , view vector v and reflection vector r !

$$l = \begin{pmatrix} 0,6 \\ 0,8 \end{pmatrix}, \quad \vec{v} = \vec{n} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad \vec{r} = -\vec{l} + 2(\vec{l} \cdot \vec{n})\vec{n} \\ = \begin{pmatrix} -0,6 \\ -0,8 \end{pmatrix} + 1,6 \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} -0,6 \\ 0,8 \end{pmatrix}$$

- b) **[3 P]** Let's assume the following Phong Lighting parameters for the surface:

$$k_{amb} = (0,0,0.2), \quad k_{diff} = (0,1,0), \quad k_{spec} = (1,0,1), \quad n_{shiny} = 2$$

The directional light is of intensity $I_{in} = (1,1,1)$.

Compute the color of point B!

$$I_{amb} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

$$L = L_{amb} + L_{diff} + L_{specular}$$

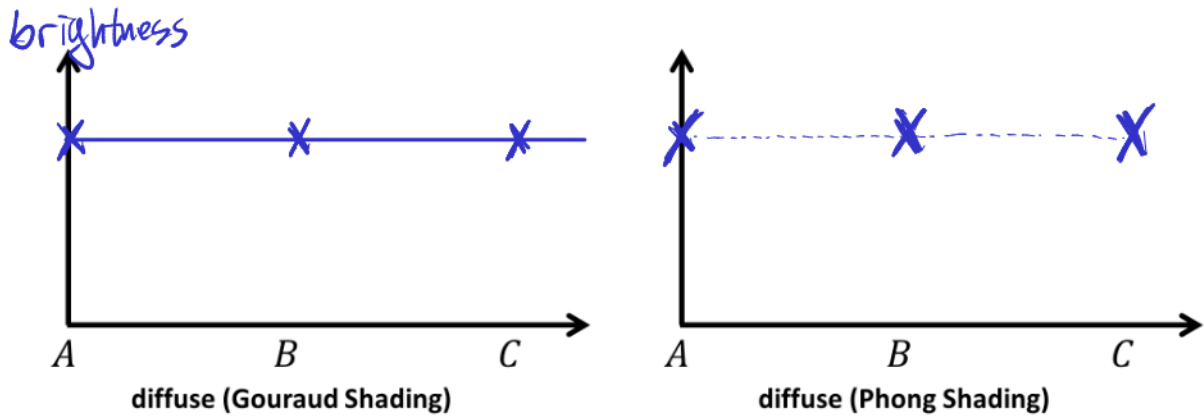
$$L_{amb} = \underset{\substack{\uparrow \\ \text{coefficient}}}{k_{amb}} + \underset{\substack{\uparrow \\ \text{intensity}}}{I_{amb}} = \begin{pmatrix} 0 \\ 0 \\ 0,2 \end{pmatrix}$$

$$L_{diff} = k_{diff} \cdot I_{in} (\vec{n} \cdot \vec{l}) = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \cdot 0,8 = \begin{pmatrix} 0 \\ 0,8 \\ 0 \end{pmatrix}$$

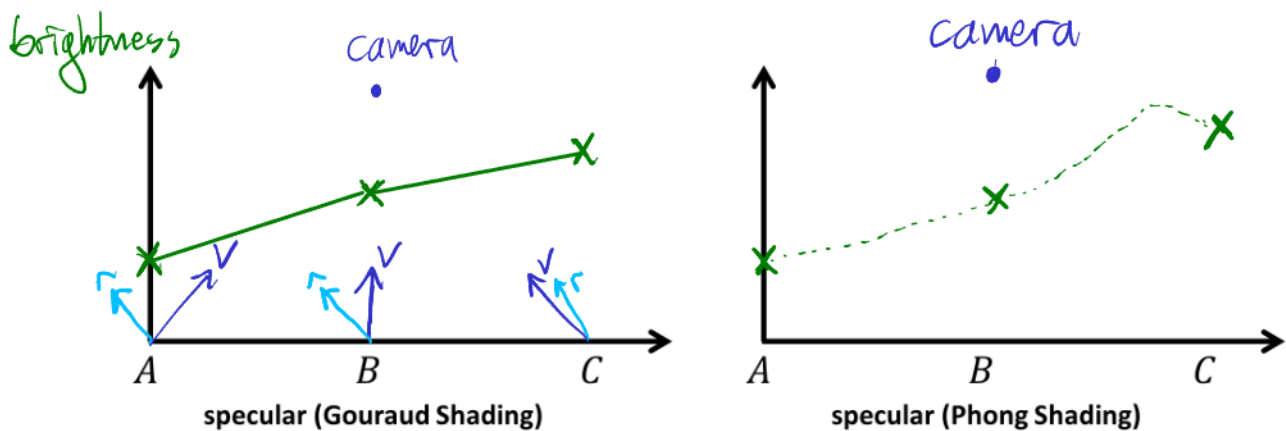
$$L_{spec} = k_{spec} \cdot I_{in} (\vec{v} \cdot \vec{r})^{n_{shiny}} = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} \cdot 0,8^2 = \begin{pmatrix} 0,64 \\ 0 \\ 0,64 \end{pmatrix}$$

- c) **[4 P]** Next, assume that both k_{diff} and k_{spec} equal $(1,1,1)$, i.e. the surface color is white in both diffuse and specular part. For the four plots below, only take into account the intensity in a qualitative sense. **You do not have to compute exact values, only a qualitative sketch is needed!**

First, sketch the **diffuse** lighting part for the surface in the picture above, using **Gouraud** shading on the left and **Phong** shading on the right!



Second, sketch the **specular** lighting part for this surface, again using **Gouraud** shading on the left and **Phong** shading on the right!





Assignment 5: Color Models [9 points]

The table below shows a list of statements concerning the color models **RGB**, **CMY**, **CMYK** and **HSV**. For each line, fill the right column with the names of all of the four-color models for which the statement is true.

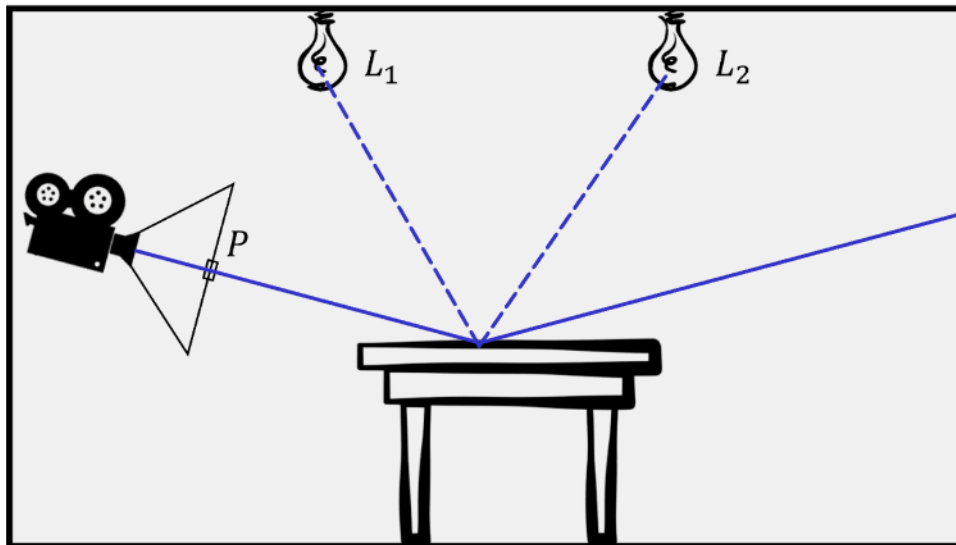
Be careful, it is also possible that **more than one** model should be filled in, or none of them at all! If the statement on the left does not apply to any of the aforementioned models, fill in "**none**" instead.

Statement	Color Model(s)
$(1, 1, 0) = \text{blue}$	CMY
subtractive color mixing	CMY, CMYK
$(0.5, 1, 1)_{\text{CMY}} = (0, 0.5, 0.5, 0.5) = \text{dark red}$	CMYK
$(1, 1, 1) = \text{black}$	CMY
perception-driven	HSV
$(0, 1, 1) = \text{cyan}$	RGB
$(1, 0, 0) = \text{green}$	none
yellow + magenta = red	CMY, CMYK
$(0.5, 0.5, 0.5) = \text{grey}$	RGB, CMY
used in monitors	RGB
$(60^\circ, 1, 1) = \text{yellow}$	HSV
cube of three primary colors	RGB, CMY

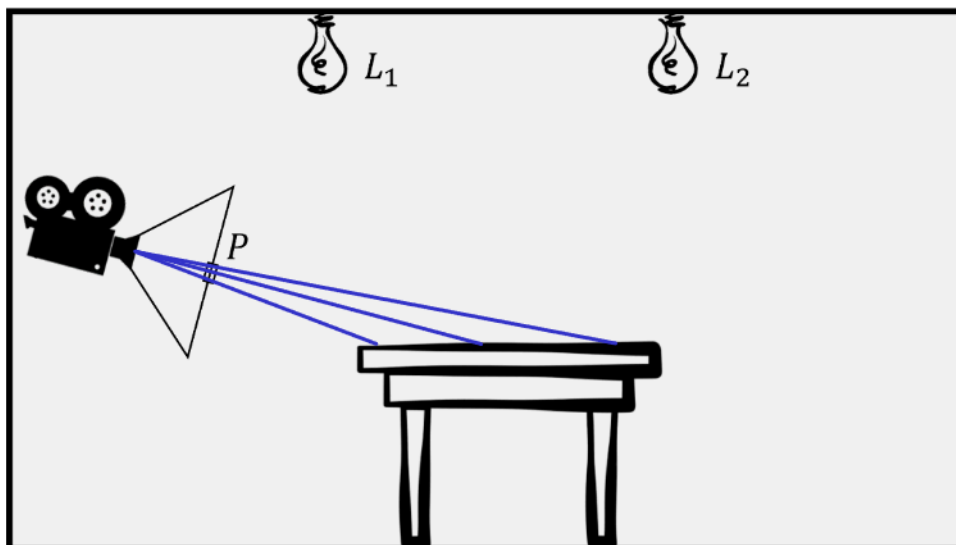
Assignment 6: Raytracing [5 points]

Given is a scene with two light sources L_1 and L_2 . We want to render the scene using different raytracing algorithms. Sketch possible rays traced through the pixel P for the respective algorithm, while using **solid** lines for **primary** and secondary rays, and **dashed** lines for **shadow** rays.

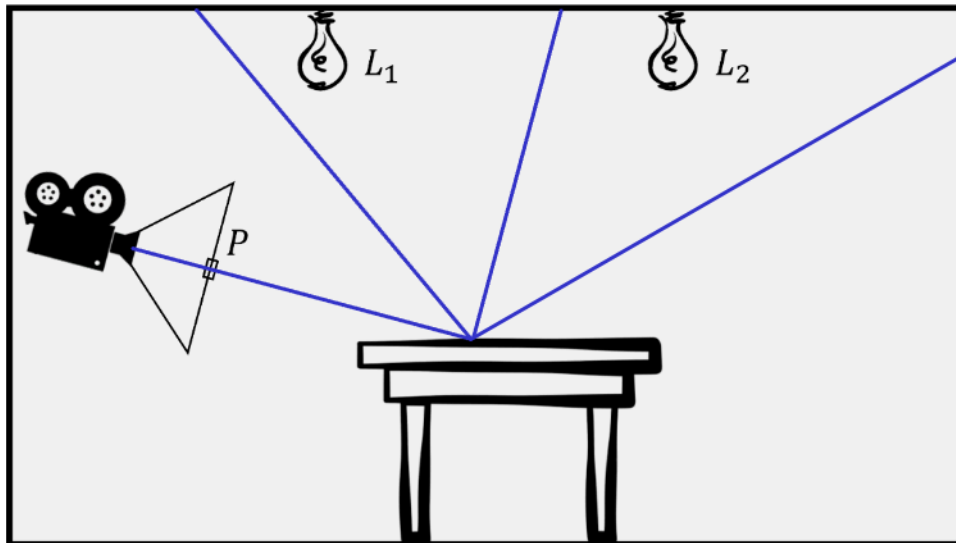
- a) [2 P] Sketch **one** eye ray through the pixel P and the respective reflection and shadow rays:



- b) [1 P] Sketch **three** possible eye rays for anti-aliasing of the pixel P :



- c) [1 P] Sketch **one** eye ray through the pixel P and **three** possible **reflection** rays for distribution ray tracing, assuming the table has a **diffuse** surface:



- d) [1 P] Sketch **one** eye ray through the pixel P and **three** possible **reflection** rays for distribution ray tracing assuming the table has a **glossy** surface:

